UT3 · Pruebas unitarias con PHPUnit

Casos, aserciones, servicios y dependencias controladas

Desarrollo web en entorno servidor · 2.º DAW

Juan Tamboleo López-Brea
Departamento de Informática
IES Ramón Arcas Meca · Lorca
Curso 2026/2027

Índice de la presentación

03–17
Preparar y ejecutar
Entorno, configuración, conceptos y primera prueba.

18–33
Diseñar casos
Aserciones, excepciones y proveedores.

34–45
Preparar y aislar
Preparación, servicios y dobles de prueba.

46–48
Mantener las pruebas
Regresión y lectura de cobertura.

Qué tipo de prueba estamos haciendo

PREPARAR Y EJECUTAR

Comprobación	Qué ejecuta	Ejemplo
Unitaria	Una unidad con dependencias controladas	Validator devuelve un error.
Integración	Colaboración real entre componentes	Subida mediante una petición HTTP.
Manual	Una persona maneja la aplicación	Enviar el formulario en el navegador.

En este bloque empezamos por unidades de código y resultados repetibles.

Elegir un comportamiento observable

PREPARAR Y EJECUTAR

Comportamiento	Dato inicial	Resultado esperado
Construir Libro	Dune, Herbert, 412	El título consultado es Dune.
Prestar	Libro disponible	estaDisponible() devuelve false.
Prestar dos veces	Libro ya prestado	Se lanza LibroNoDisponibleException.
Validar título	Tres espacios	Se devuelve el mensaje de error.

La expectativa procede del contrato del programa, no de copiar su implementación.

Qué PHP usará PHPUnit

PREPARAR Y EJECUTAR

PHPUnit ejecuta las pruebas con PHP desde la terminal (CLI). Primero comprobamos qué PHP está usando esa terminal.

Comando	Qué muestra	Para qué lo consultamos
`php -v`	Versión de PHP y modo CLI.	Comprobar PHP 8.3 o superior para PHPUnit 12.5.
`php --ini`	Archivo php.ini cargado y archivos adicionales.	Localizar la configuración que debemos revisar.
`php -m`	Módulos y extensiones cargados.	Comprobar extensiones requeridas, como mbstring o dom.

Se escriben por separado en la terminal. El PHP del servidor web puede utilizar otra configuración.

Instalar como dependencia de desarrollo

PREPARAR Y EJECUTAR

Instalamos PHPUnit en el proyecto para escribir y ejecutar sus pruebas. --dev registra el paquete entre las herramientas de desarrollo.

```
composer require --dev "phpunit/phpunit:^12.5"
```

Paquete local

Composer instala PHPUnit y sus dependencias dentro de vendor.

require-dev

Añade phpunit/phpunit a require-dev en composer.json. ^12.5 permite versiones desde 12.5 y anteriores a 13.0.

En otra copia

Con composer.lock existente, composer install recupera las versiones registradas, incluidas las de desarrollo por defecto.

Para qué sirve phpunit.xml

Es el archivo de configuración de PHPUnit. Guarda cómo iniciar las pruebas y dónde encontrarlas, para no indicar esas opciones en cada ejecución.

Elemento	Función
`bootstrap`	Ejecuta un archivo de preparación antes de los tests. Usamos el autoload de Composer.
`testsuites / testsuite`	Agrupan las pruebas. Biblioteca es el nombre elegido para el grupo.
`directory`	Indica la carpeta que PHPUnit recorrerá buscando archivos de prueba.

Se guarda junto a composer.json. Al ejecutar PHPUnit desde esa raíz, se lee automáticamente.

Ejemplo de configuración en phpunit.xml

PREPARAR Y EJECUTAR

```
<phpunit bootstrap="vendor/autoload.php">
  <testsuites>
    <testsuite name="Biblioteca">
      <directory>tests</directory>
    </testsuite>
  </testsuites>
</phpunit>
```

Antes de los tests

vendor/autoload.php registra la carga de las clases de Composer y del proyecto.

Dónde busca

En tests, que contendrá archivos como LibroTest.php. src contiene las clases de la aplicación.

Nombre del grupo

Biblioteca identifica esta suite. No es una clase PHP ni una carpeta.

Qué significa composer test

PREPARAR Y EJECUTAR

Composer permite guardar comandos con un nombre en scripts. test es el nombre que elegimos para ejecutar PHPUnit.

```
"scripts": {
    "test": "phpunit"
}
```

Configuración

Este fragmento se añade al objeto de composer.json junto a los apartados existentes.

Ejecución

composer test pide a Composer que ejecute el comando asociado a test.

Ejecutable local

Composer encuentra phpunit en vendor/bin, instalado como dependencia del proyecto.

Qué contiene una prueba de PHPUnit

PREPARAR Y EJECUTAR

Una prueba automatizada ejecuta un comportamiento y comprueba si cumple una expectativa. PHPUnit descubre y ejecuta los métodos de prueba.

Elemento	Función
`tests/LibroTest.php`	Archivo de prueba. Su nombre termina en Test.php.
`extends TestCase`	Hereda la clase base de PHPUnit y sus métodos de comprobación.
`public function test…`	Método que PHPUnit reconoce como prueba. En el primer ejemplo no recibe argumentos y devuelve void.
Aserción	Comprobación del resultado. Si falla, PHPUnit informa del fallo y termina ese método.

Arrange, Act, Assert

PREPARAR Y EJECUTAR

Estas tres fases organizan una prueba para que se distinga qué necesita, qué ejecuta y qué comprueba.

Fase	Qué hacemos
Arrange · Preparar	Crear los datos, objetos y condiciones iniciales.
Act · Ejecutar	Llamar a la operación cuyo comportamiento queremos comprobar.
Assert · Comprobar	Comparar el resultado observable con la expectativa del contrato.

Primero se decide el resultado esperado. Después se escribe la aserción que lo verifica.

assertSame compara tipo y valor

PREPARAR Y EJECUTAR

Una aserción expresa una condición que debe cumplirse. assertSame exige el mismo tipo y valor. Si se comparan objetos, exige la misma instancia.

```
$this->assertSame($esperado, $obtenido);
```

Primer argumento

El valor esperado según el contrato del programa.

Segundo argumento

El resultado obtenido al ejecutar el código.

$this

Es el objeto de la clase de prueba, que hereda las aserciones de TestCase.

Primera clase de prueba completa

PREPARAR Y EJECUTAR

```php
<?php
use App\Model\Libro;
use PHPUnit\Framework\TestCase;
final class LibroTest extends TestCase
{
    public function testConservaTitulo(): void
    {
        $libro = new Libro('Dune', 'Frank Herbert',
            412);
        $this->assertSame('Dune', $libro->titulo());
    }
}
```

Archivo

tests/LibroTest.php.

Método

testConservaTitulo es público, no recibe argumentos y devuelve void.

Aserción

El esperado va primero y el resultado obtenido después.

Arrange, Act, Assert en el mismo test

PREPARAR Y EJECUTAR

Preparación y acción

```
// Arrange: preparar la entrada
$titulo = 'Dune';
// Act: ejecutar la construcción
$libro = new Libro($titulo,
    'Frank Herbert', 412);
```

Comprobación

```
// Assert: comprobar lo observable
$this->assertSame(
    'Dune', $libro->titulo()
);
```

Los comentarios ayudan al principio. La estructura debe seguir siendo legible sin repetir etiquetas en cada línea.

Ejecutar la primera prueba

PREPARAR Y EJECUTAR

Desde la raíz del proyecto, composer test ejecuta el script definido en composer.json. PHPUnit lee phpunit.xml y ejecuta la prueba de tests/LibroTest.php.

Comando

```
composer test
```

Resultado esperado

```
.
OK (1 test, 1 assertion)
```

Un punto indica un test correcto. Este resultado corresponde a ejecutar solo la primera prueba.

Ejecutar PHPUnit sin el script de Composer

PREPARAR Y EJECUTAR

Si no se ha definido el script test, podemos ejecutar directamente el archivo de PHPUnit instalado dentro del proyecto.

```
php vendor/bin/phpunit
```

php	vendor/bin/phpunit	Misma configuración
Abre el intérprete de PHP de la terminal.	Es el ejecutable local que Composer ha instalado para PHPUnit.	Desde la raíz, también lee phpunit.xml y ejecuta las pruebas configuradas.

Con el script "test": "phpunit", ambas formas ejecutan PHPUnit. Instalar el paquete no crea por sí solo el script test.

Leer un fallo de aserción

PREPARAR Y EJECUTAR

Expectativa incorrecta

```
$libro = new Libro('Dune',
    'Frank Herbert', 412);
$this->assertSame('Solaris',
    $libro->titulo());
```

Diferencia informada

```
F
Failed asserting that two strings
    are identical.
- 'Solaris'
+ 'Dune'
```

El fallo obliga a revisar contrato, entrada y resultado; no a cambiar el esperado para obtener verde sin investigar.

Elegir entre assertSame y assertEquals

DISEÑAR CASOS

Aserción	Qué exige
`assertSame($esperado, $obtenido)`	Mismo tipo y valor. En objetos, la misma instancia.
`assertEquals($esperado, $obtenido)`	Igualdad según las reglas de comparación de PHPUnit. Puede aceptar tipos distintos.

Ambas reciben primero el esperado y después el obtenido. Admiten un mensaje opcional como tercer argumento.

Ejemplos de comparación

DISEÑAR CASOS

```
$this->assertSame(412, 412);     // pasa
$this->assertSame(412, '412');   // falla
$this->assertEquals(412, '412'); // pasa
```

Mismo tipo

412 es un entero en ambos lados de la primera comparación.

Tipo distinto

La cadena '412' no es idéntica al entero 412.

En el modelo

assertSame detecta si paginas() devuelve una cadena cuando se espera un entero.

Aserciones de booleanos y null

DISEÑAR CASOS

Estas aserciones reciben el resultado obtenido. Comprueban el valor exacto, sin convertirlo a otro tipo.

Aserción	Condición que debe cumplirse
`assertTrue($obtenido)`	El resultado es exactamente el booleano true.
`assertFalse($obtenido)`	El resultado es exactamente el booleano false.
`assertNull($obtenido)`	El resultado es exactamente null. No equivale a false, 0 o una cadena vacía.

Ejemplo de disponibilidad y ausencia

DISEÑAR CASOS

```
$libro = new Libro('Dune', 'Frank Herbert', 412);
$this->assertTrue($libro->estaDisponible());

$libro->prestar();
$this->assertFalse($libro->estaDisponible());

$repo = new ArrayLibroRepository();
$this->assertNull($repo->buscarPorId(999));
```

El libro empieza disponible y deja de estarlo tras prestar. El repositorio nuevo devuelve null porque no contiene el ID 999.

Aserciones sobre colecciones

DISEÑAR CASOS

Elegimos la aserción según la propiedad que queremos comprobar. Los argumentos esperados se escriben antes que la colección obtenida.

Aserción	Qué comprueba
`assertCount($cantidad, $coleccion)`	El array u objeto contable tiene exactamente esa cantidad de elementos.
`assertArrayHasKey($clave, $array)`	Existe esa clave en el array, aunque su valor sea null.
`assertSame([], $array)`	El resultado es exactamente un array vacío.

assertCount comprueba la cantidad. No comprueba cuáles son los elementos ni su contenido.

Ejemplos con arrays

DISEÑAR CASOS

```
$libros = ['Dune', 'Solaris'];
$this->assertCount(2, $libros);

$errores = ['titulo' => 'Escribe el título.'];
$this->assertArrayHasKey('titulo', $errores);

$errores = [];
$this->assertSame([], $errores);
```

Son tres comprobaciones ilustrativas. En las pruebas del programa se usan los arrays que devuelve la operación probada.

Construcción y lectura del modelo

DISEÑAR CASOS

```
public function testConservaDatos(): void
{
    $libro = new Libro('Dune', 'Frank Herbert',
        412);
    $this->assertSame('Dune', $libro->titulo());
    $this->assertSame('Frank Herbert',
        $libro->autor());
    $this->assertSame(412, $libro->paginas());
}
```

Contrato

Se comprueba el estado observable creado con una entrada válida.

Alcance

Estos valores pertenecen a una misma construcción.

Continuidad

El constructor es el mismo que en formularios.

Una operación cambia el estado

DISEÑAR CASOS

```
public function testPrestarCambiaDisponibilidad():
    void
{
    $libro = new Libro('Dune', 'Frank Herbert',
        412);
    $libro->prestar();
    $this->assertFalse($libro->estaDisponible());
}
```

Arrange

El constructor deja el libro disponible.

Act

prestar ejecuta el cambio.

Assert

Se consulta el estado mediante la API pública.

Cómo se comprueba una excepción

DISEÑAR CASOS

Algunas operaciones deben rechazar una situación lanzando una excepción. expectException declara qué clase de excepción esperamos.

```
$this->expectException(ClaseExcepcion::class);
```

Orden	Acción
1 · Preparar	Crear el estado que debe provocar el rechazo.
2 · Declarar la expectativa	Indicar la excepción justo antes de la operación que debe lanzarla.
3 · Ejecutar	Llamar a esa operación. PHPUnit comprueba si lanza la excepción esperada.

Si no se lanza la excepción, la prueba falla. La expectativa se declara antes de ejecutar la operación.

Esperar una excepción

DISEÑAR CASOS

```
expectException(string $exception): void
```

```
$libro = new Libro('Dune', 'Frank Herbert', 412);
$libro->prestar();
$this->expectException(
    LibroNoDisponibleException::class);
$libro->prestar();
```

Orden

Preparar, declarar la expectativa y ejecutar la operación que debe fallar.

Resultado

Pasa si esa operación lanza la excepción esperada.

Importación

LibroNoDisponibleException pertenece al mismo modelo del ejemplo.

Casos del formulario antes de escribir tests

DISEÑAR CASOS

Entrada de páginas	Resultado	Qué cubre
"1"	Válida	Mínimo permitido.
"3000"	Válida	Máximo permitido.
"0" y "3001"	Error	Fuera del rango.
"12abc"	Error	Formato incorrecto.
["412"]	Error	Tipo inesperado.

Añadir casos de título vacío, longitud y fecha imposible a sus reglas correspondientes.

Qué es un proveedor de datos

DISEÑAR CASOS

Un proveedor permite repetir el mismo test con entradas diferentes. Devuelve filas de argumentos y PHPUnit ejecuta el método de prueba una vez por fila.

Elemento	Cómo se relaciona con la prueba
Método proveedor	Es public static. En estos ejemplos devuelve un array de casos.
Cada fila	Contiene los argumentos, en el orden de los parámetros del test.
`#[DataProvider(...)]`	Atributo colocado antes del test. Identifica qué proveedor entrega sus argumentos.

Los nombres de las filas permiten identificar cada caso en el informe de PHPUnit. El proveedor no utiliza $this.

Proveedor de datos: declarar los casos

DISEÑAR CASOS

```
public static function titulosVacios(): array
{
    return [
        'vacío' => [''],
        'espacios' => ['   '],
        'tabulación' => ["\t"],
    ];
}
```

public static

PHPUnit obtiene los casos sin utilizar una instancia del test.

Cada fila

Contiene los argumentos que recibirá el método de prueba.

Nombres

vacío y espacios identifican el caso en el informe.

Proveedor de datos: utilizar los casos

DISEÑAR CASOS

```php
use PHPUnit\Framework\Attributes\DataProvider;
#[DataProvider('titulosVacios')]
public function testTituloRequerido(string $titulo):
    void
{
    $v = new Validator();
    $this->assertSame(
        'Escribe el título.',
        $v->requerido($titulo, 'Escribe el título.')
    );
}
```

Atributo

#[DataProvider(...)] asocia el método con sus datos.

Ejecución

Se ejecuta una prueba por fila del proveedor.

Independencia

El proveedor es estático y no utiliza el estado del objeto de prueba.

Proveedor de páginas y límites

DISEÑAR CASOS

```
public static function entradasPaginas(): array
{
    return [
        'mínimo' => ['1', true],
        'máximo' => ['3000', true],
        'bajo' => ['0', false],
        'alto' => ['3001', false],
        'formato' => ['12abc', false],
        'array' => [['412'], false],
    ];
}
```

Argumentos

La entrada y si debe aceptarse.

Cobertura

Límites, formato y tipo se comprueban de forma independiente.

Probar el servicio de formulario

DISEÑAR CASOS

```
#[DataProvider('entradasPaginas')]
public function testPaginas(mixed $entrada, bool
    $valida): void
{
    $post = $this->formularioValido();
    $post['paginas'] = $entrada;
    $formulario = new LibroFormService(
        new Validator());
    $r = $formulario->procesar($post);
    $this->assertSame(
        $valida, !isset($r['errores']['paginas'])
    );
}
```

Base conocida

formularioValido devuelve título Dune, autor Frank Herbert, páginas 412, género ciencia y fecha 2026-10-24.

Sustitución

Solo cambia páginas para aislar el caso estudiado.

Resultado

Cada fila ejecuta procesar y comprueba el error correspondiente.

Preparar y limpiar cada prueba

PREPARAR Y AISLAR

La fixture es el conjunto de datos, objetos y recursos que necesita una prueba. Cada test debe poder ejecutarse de forma independiente.

Método	Cuándo se ejecuta y qué prepara
`setUp(): void`	PHPUnit lo llama antes de cada prueba. Permite crear una preparación común.
`tearDown(): void`	PHPUnit lo llama después de cada prueba. Permite limpiar recursos creados por ella.

En estos ejemplos ambos métodos son protected. Un objeto nuevo en setUp evita que un test herede el estado dejado por otro.

Cada prueba prepara su propio estado

PREPARAR Y AISLAR

Prueba de prestar

```
// Una prueba nueva
$libro = new Libro('Dune',
    'Frank Herbert', 412);
$libro->prestar();
$this->assertFalse(
    $libro->estaDisponible());
```

Prueba del estado inicial

```
// Otra prueba nueva
$libro = new Libro('Dune',
    'Frank Herbert', 412);
$this->assertTrue(
    $libro->estaDisponible());
```

Ninguna necesita que otra prueba se ejecute antes.

setUp prepara cada prueba

PREPARAR Y AISLAR

```
private Libro $libro;
protected function setUp(): void
{
    $this->libro = new Libro('Dune',
        'Frank Herbert',
        412);
}
public function testInicialmenteDisponible(): void
{
    $this->assertTrue(
        $this->libro->estaDisponible());
}
```

Ciclo de vida

PHPUnit ejecuta setUp antes de cada test.

Reutilización

Evita repetir una preparación que varios tests necesitan.

Estado nuevo

El objeto se reconstruye; una prueba no hereda el préstamo de otra.

tearDown limpia recursos externos

PREPARAR Y AISLAR

```
private string $temporal;

protected function tearDown(): void
{
    if (isset($this->temporal)
        && is_file($this->temporal)) {
        unlink($this->temporal);
    }
}
```

Cuándo

Si la prueba crea un archivo temporal, se retira después.

No hace falta

No es necesario para liberar cada objeto Libro.

Control

La ruta pertenece al test y se prepara con tempnam; no procede del usuario.

El servicio que vamos a probar

PREPARAR Y AISLAR

```
final class BibliotecaService
{
    public function __construct(private
        LibroRepository $repo) {}
    public function prestar(int $id): void
    {
        $libro = $this->repo->buscarPorId($id);
        if ($libro === null) {
            throw new
                OutOfBoundsException(
                    'Libro no encontrado.');
        }
        $libro->prestar();
    }
}
```

Colaboración

El repositorio localiza el libro; el objeto realiza el préstamo.

Casos

Libro existente, inexistente y ya prestado.

Servicio con repositorio en memoria

PREPARAR Y AISLAR

```
$repo = new ArrayLibroRepository();
$id = $repo->guardar(
    new Libro('Dune', 'Frank Herbert', 412)
);
$servicio = new BibliotecaService($repo);

$servicio->prestar($id);

$this->assertFalse(
    $repo->buscarPorId($id)->estaDisponible()
);
```

Preparación

Repositorio nuevo con un libro conocido.

Servicio

BibliotecaService localiza el libro y llama a prestar.

Alcance

Aquí se comprueba su colaboración con el repositorio en memoria.

Qué es un doble de prueba

PREPARAR Y AISLAR

Un doble sustituye un colaborador real durante una prueba. Así podemos controlar sus respuestas y comprobar el servicio en condiciones conocidas.

Tipo	Qué aporta
Stub	Entrega respuestas preparadas para provocar una situación concreta en el servicio.
Mock	Permite comprobar interacciones esperadas, como el número de llamadas y sus argumentos.

Se crea un doble de LibroRepository y se inyecta en BibliotecaService. El servicio que se prueba sigue siendo real.

Sintaxis de un stub

PREPARAR Y AISLAR

Expresión	Recibe	Devuelve o configura
`createStub(Interfaz::class)`	Nombre de la interfaz	Doble compatible con ella.
`$doble->method("buscarPorId")`	Nombre del método	Configurador de su respuesta.
->willReturn(null)	Respuesta prevista	Configura que devuelva null.

Este stub controla la entrada indirecta del servicio; no exige una cantidad de llamadas.

Stub: controlar lo que devuelve el repositorio

PREPARAR Y AISLAR

```
$repo = $this->createStub(LibroRepository::class);
$repo->method('buscarPorId')->willReturn(null);
$servicio = new BibliotecaService($repo);

$this->expectException(OutOfBoundsException::class);
$servicio->prestar(999);
```

Entrada indirecta	Objetivo	API nueva
El repositorio siempre informa de que no hay libro.	Comprobar cómo responde el servicio a esa situación.	createStub crea el doble de la interfaz; method selecciona el método y willReturn fija su respuesta.

Sintaxis de un mock

PREPARAR Y AISLAR

Expresión	Qué indica
`createMock(Interfaz::class)`	Crea un doble que permite expectativas.
`expects($this->once())`	Esperar exactamente una llamada.
`method("buscarPorId")`	Método cuya llamada se comprueba.
with(1)	Argumento que debe recibir.
`willReturn($libro)`	Respuesta utilizada durante la prueba.

Estas llamadas preparan la expectativa antes de ejecutar el servicio.

Mock: comprobar una interacción relevante

PREPARAR Y AISLAR

```
$libro = new Libro('Dune', 'Frank Herbert', 412);
$repo = $this->createMock(LibroRepository::class);
$repo->expects($this->once())
    ->method('buscarPorId')->with(1)
    ->willReturn($libro);

(new BibliotecaService($repo))->prestar(1);
$this->assertFalse($libro->estaDisponible());
```

Expectativa

La consulta se realiza una vez y con el ID 1.

Diferencia

El stub controla una respuesta; este mock verifica además la llamada.

Límite

Se dobla la interfaz del repositorio. Se utiliza un Libro real, aunque su clase sea final.

Validar archivos y probar una subida

PREPARAR Y AISLAR

Prueba	Preparación	Qué demuestra
Unitaria de regla	Tamaño 2 MiB + 1	Se rechaza por tamaño.
Unitaria de regla	MIME text/plain	Se rechaza por tipo.
Integración HTTP	Enviar un multipart real	Recepción y movimiento del archivo.
Fichero local desde CLI	Crear un temporal cualquiera	No simula una subida HTTP válida.

Separar la decisión de validación del movimiento facilita comprobar cada responsabilidad.

Seleccionar pruebas y conservar regresiones

MANTENER LAS PRUEBAS

Concepto	Para qué sirve
Filtro de ejecución	--filter selecciona pruebas por su nombre. Sirve para ejecutar un caso concreto mientras se investiga.
Prueba de regresión	Reproduce un defecto y se conserva después de corregirlo para detectar si vuelve a aparecer.
Argumentos de PHPUnit	En composer test -- --filter nombre, el separador -- indica que lo siguiente debe llegar al comando del script.

Seleccionar un test y fijar una regresión

MANTENER LAS PRUEBAS

```
composer test -- --filter testTituloRequerido

# Caso conservado en el proveedor:
# espacios => error de título requerido
```

Selección

--filter ejecuta una selección de métodos.

Regresión

Añadir el caso que reproduce el defecto, comprobar el fallo y corregir la regla.

Diagnóstico

Una clase no encontrada es un error de ejecución, distinto de una aserción fallida.

Qué aporta la cobertura

Pregunta	Respuesta
¿Qué informa?	Qué partes del código se ejecutaron al correr los tests.
¿Qué no demuestra?	Que las expectativas sean correctas o que todos los casos estén comprobados.
¿Qué requiere?	Un controlador como Xdebug o PCOV, configurado en PHP CLI.
Ejemplo	Un test puede ejecutar prestar sin comprobar la disponibilidad: ejecuta código y todavía verifica poco.

La cobertura complementa la selección de casos. No se fija un porcentaje mínimo en esta unidad.